University of Massachusetts, Lowell
Department of Chemical Engineering

MIMO RC Circuit Control

Patrick Heng

CHEN 5300

Prof. E.L. Maase

05/04/26

Contents

GITHUB LINK TO CODE

Introduction

The following report studies the dynamics of an analog multiple input, multiple output (MIMO) RC circuit. The circuit was implemented on an Arduino board which served as the voltage sensor and transmitter. The controller was a Python script which implemented PID and model predictive control (MPC) algorithms. Using this apparatus, the first objective of the study was to develop an empirical model of the circuit and design algorithms to control the capacitor voltages by manipulating the voltage sources. With these algorithms, the report then compares PID and MPC control schemes for setpoint tracking and disturbance rejection. Overall, it was concluded that MPC only yields marginal benefits over PID control for weakly interacting systems and small disturbances. However, MPC excels at controlling strongly interacting systems with large and frequent disturbances.

System Modeling

Theory

RC circuits consist of only resistors and capacitors paired with time-varying voltage or current sources. The circuits are governed by Ohm's law and the capacitor charging equation,

$$\underbrace{V_R(t) = RI(t), \quad V_C(t) = \frac{1}{C}\int_0^t I(\tau)d\tau}_{\text{Time}} \iff \underbrace{V_R(s) = RI(s), \quad V_C(s) = \frac{1}{Cs}I(s),}_{\text{Laplace}} \tag{1}$$

where V_R and V_C are the voltages across the resistor and capacitor, I is the current, R is the resistance, and C is the capacitance. When paired with Kirchhoff's laws, a general RC circuit can be reformulated in terms of impedances, $Z_{ij}(s)$, by the following matrix equation,

$$\mathbf{V}_S = \mathbf{ZI} \iff \begin{pmatrix} V_1 \\ V_2 \\ V_3 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} Z_{11} & Z_{12} & Z_{13} & Z_{14} & Z_{15} \\ Z_{21} & Z_{22} & Z_{23} & Z_{24} & Z_{25} \\ Z_{31} & Z_{32} & Z_{33} & Z_{34} & Z_{35} \\ Z_{41} & Z_{42} & Z_{43} & Z_{44} & Z_{45} \\ Z_{51} & Z_{52} & Z_{53} & Z_{54} & Z_{55} \end{pmatrix} \begin{pmatrix} I_1 \\ I_2 \\ I_3 \\ I_4 \\ I_5 \end{pmatrix}. \tag{2}$$

where $\mathbf{V}_S$ is the vector of source voltages and the I_j are currents around the circuit loops. Once the currents around each loop are known, the voltage across any component can be solved for. For example, for the voltages across the capacitors, $\mathbf{V}_C$,

$$\mathbf{V}_C = \mathbf{GV}_S, \quad \mathbf{G} = \mathbf{G}(\mathbf{I}, \mathbf{Z}), \tag{3}$$

where $\mathbf{G}$ is a transfer function that depends on the currents and impedances. The objective of the project is to identify $\mathbf{G}$ and use this model to help control the voltages across the capacitors by manipulating the voltage sources. This is akin to level control in coupled tanks by controlling the pressure drop across control valves.

While a theoretical model of $\mathbf{G}$ exists[1], it is likely a high order transfer function that is difficult to work with. Therefore, a system identification procedure must be performed to

[1]This would require symbolic inversion of a 5×5 transfer function matrix... I tried it. I gave up.

fit a simpler model to each G_{ij}. This can be done by inputting a step voltage to one of the sources and measuring the resulting capacitor voltages. This can be repeated for each source to identify the full model. Alternatively, a frequency response method could be used by inputting multiple sine inputs at different frequencies. However, this was found to be difficult to implement in practice.

With the system model identified, different control schemes will be tested and compared. The simplest scheme is PID control with CV-MV parings derived by relative gain array (RGA) analysis. Model predictive control (MPC) more complex, but can explicitly account for the interactions and constraints of the system. The focus of this report will be on these control laws, which will be compared with step setpoint changes and random disturbance inputs. The implementation of these algorithms will be discussed further in the respective sections.

Apparatus

The selected RC circuit is shown in the following circuit diagram.

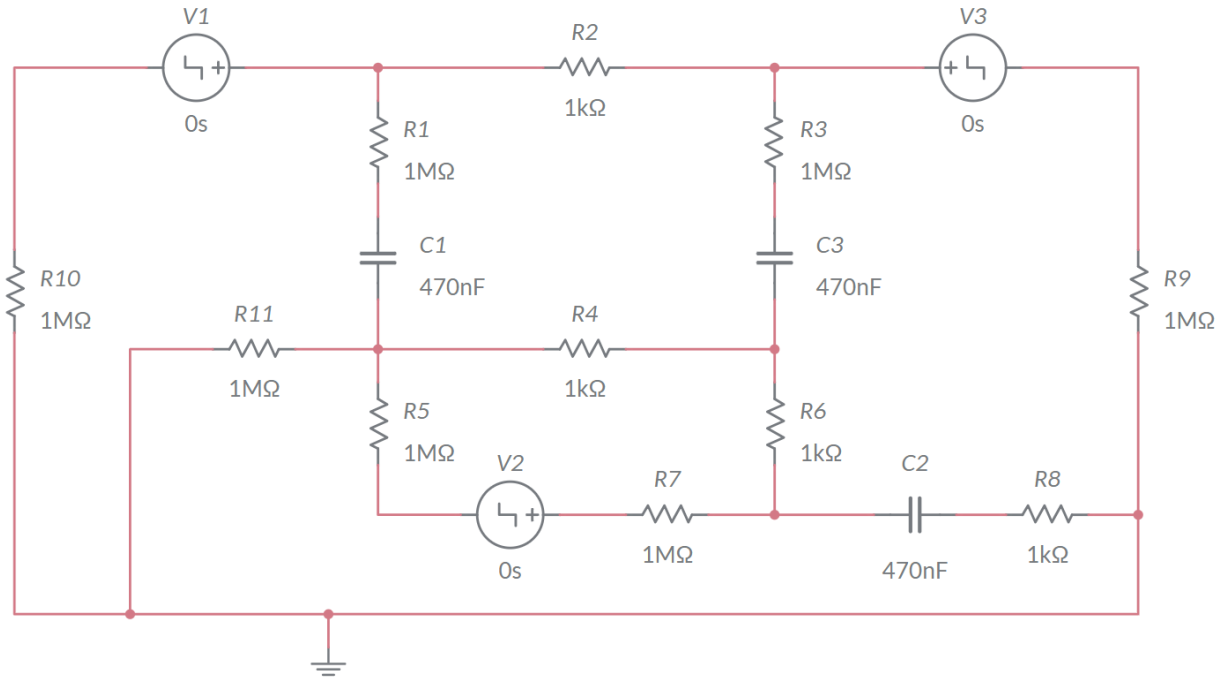

Figure 1: Schematic of RC circuit, 'V' denotes a voltage source, 'R' denotes a resistor, and 'C' denotes a capacitor

The circuit is nearly symmetric between V1/C1 and V3/C3, so it is expected that these components will give similar responses. V2 is in the middle of the circuit, meaning that it should have a high amount of interaction with all the capacitors. Conversely, V1 and V3 are in their own branches with C1 and C3, so they should interact less with each other. Note the placement of the 1kΩ and 1MΩ resistors. The large difference between these values should

give rise to fast and slow time scales for the small and large resistors respectively. The 470 nF capacitors were chosen due to the availability of components.

The circuit was implemented on a physical Arduino board, which serves as the sensor and transmitter. The controller will be a Python script, which interacts with the Arduino board in real time. The Arduino board and capacitors have the following constraints:

- $0 \text{ V} \leq V_S \leq 5 \text{ V}$
- $-5 \text{ V} \leq V_C \leq 5 \text{ V}$
- Sampling rate - Variable, 0.03 seconds average

The first constraint is that the board can supply up to 5 V to each voltage source. The second constraint states that the measured voltage across the capacitors is limited to ± 5 V. The last constraint is that the fastest sampling time is about 0.03 seconds, which is the required time for the board to read and write the voltage signals.

The basic structure of the control system is summarized as follows.

1. Initialize connection between Arduino and Python script
2. Read capacitor voltages; send to Python script
3. Python script calculates control moves for all inputs; send to Arduino
4. Arduino sets voltage sources to calculated inputs
5. Repeat steps 2-4 until the maximum time iterations have been reached

A block diagram of the system is also presented in the following figure.

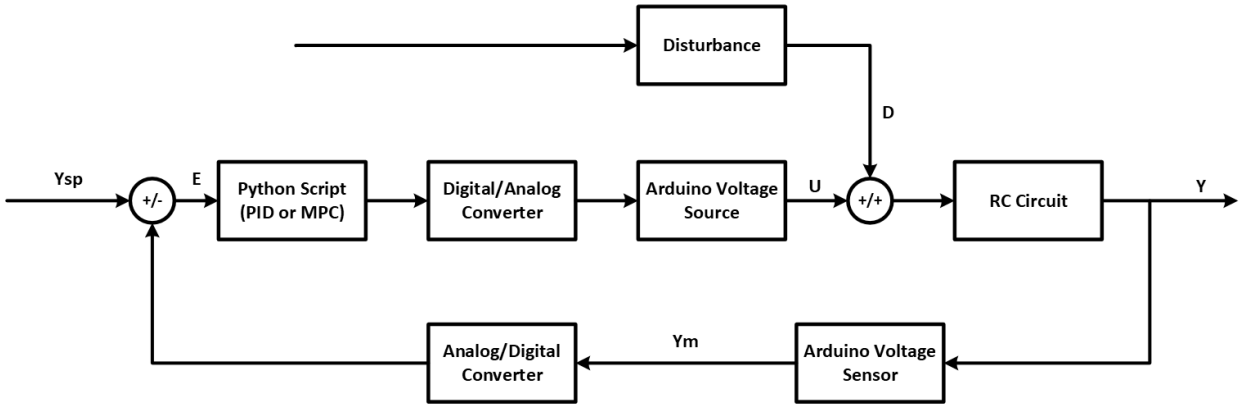

Figure 2: Block diagram of RC circuit-Arduino control system

The disturbances will be discussed in more detail later, but they are outputted by the Python script and are not measured. Since the Arduino only uses digital inputs and outputs, the analog input and output voltages need to be discretized so that they can be processed. The Arduino contains an analog to digital (A/D) converter for the measured voltages which

converts 0-5 V to an integer from 0-1023. Conversely, the analog voltage sources require a digital to analog (D/A) converter from a 0-255 integer to a 0-5 V output[2].

System Identification

As stated previously, an empirical model will be developed to characterize the effect of the voltage sources on the capacitor voltages. RC circuits are essentially perfect linear models, so transfer functions are a natural approach for the mathematical description. Let $\mathbf{u}$ denote the input voltages and $\mathbf{y}$ denote the output capacitor voltages, then the transfer function model is,

$$\Delta\mathbf{y} = \mathbf{G}\Delta\mathbf{u}, \tag{4}$$

where $\mathbf{G}$ is the system transfer function matrix. The elements, G_{ij}, of the matrix can be identified by varying input u_j and measuring the response of each y_i. For example, the first voltage source is changed by a $\Delta u_1 = 1$ V step input and the voltage responses are measured for Δy_1, Δy_2, and Δy_3. An assumed model can then be regressed for G_{11}, G_{12}, and G_{13} based on what the transient response looks like. A least squares procedure can then be used to extract the model parameters. The model can then be verified by comparing it to data from a different magnitude step test for the same input. This process is repeated for each u_j, which populates the entire $\mathbf{G}$ matrix.

Fitting the Model

Using the method outlined previously, the following data was measured for each of the step inputs. The plots show the transient response of Δy_i normalized by the input magnitude, Δu_j. The experimental data is shown as dots while the fitted model is plotted as a continuous line with the same color. For brevity, only one step response is shown for each input; validation data is presented in the appendix. The model parameters are presented at the end of this section.

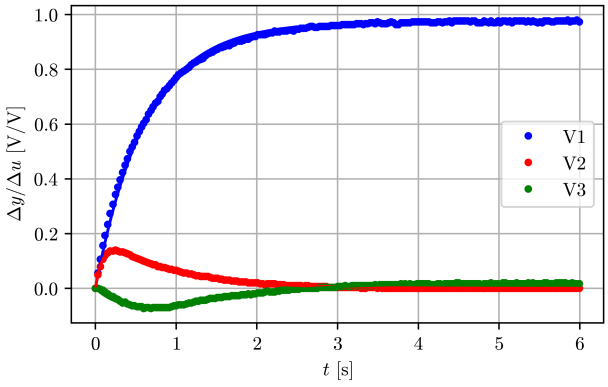

Figure 3: y_1, y_2, y_3 response to $u_1 = 0 \rightarrow 3$ V step input

[2]This is done with pulse width modulation (PWM).

The $\Delta y_1/\Delta u_1$ response appears to be a first order model with a time constant of $\tau = 0.8$ s and a gain of $K = 1$ V/V. Mathematically this gives,

$$\frac{\Delta y_1(t)}{\Delta u_1(t)} = K\left(1 - e^{-t/\tau}\right) \iff \frac{\Delta Y_1(s)}{\Delta U_1(s)} = \frac{K}{\tau s + 1}. \tag{5}$$

The $\Delta y_2/\Delta u_1$ response has a peak due to the step input, but settles back down to 0. This can the thought of the difference between two exponential decays, one with a slow time scale and one with a fast time scale. This gives the following step response model,

$$\frac{\Delta y_2(t)}{\Delta u_1(t)} = K\left(e^{-t/\tau_1} - e^{-t/\tau_2}\right) \iff \frac{\Delta Y_2(s)}{\Delta U_1(s)} = K\left[\frac{\tau_1 s}{\tau_1 s + 1} - \frac{\tau_2 s}{\tau_2 s + 1}\right]. \tag{6}$$

The $\Delta y_3/\Delta u_1$ pair has an inverse response and settles slightly above 0. The simplest transfer function with this behavior is a second order model with a positive zero. This gives the following step response model,

$$\frac{\Delta y_3(t)}{\Delta u_1(t)} = K\left[1 - \left(\frac{\tau_3 - \tau_1}{\tau_2 - \tau_1}\right)e^{-t/\tau_1} + \left(\frac{\tau_3 - \tau_2}{\tau_2 - \tau_1}\right)e^{-t/\tau_2}\right], \tag{7}$$

$$\iff \frac{\Delta Y_3(s)}{\Delta U_1(s)} = \frac{K(\tau_3 s + 1)}{(\tau_1 s + 1)(\tau_2 s + 1)}. \tag{8}$$

where $\tau_3 < 0 < \tau_1, \tau_2$. Overall, the u_1 input has small interactions with the y_2 and y_3, while having a large effect on y_1. This suggests the pairing of u_1 and y_1.

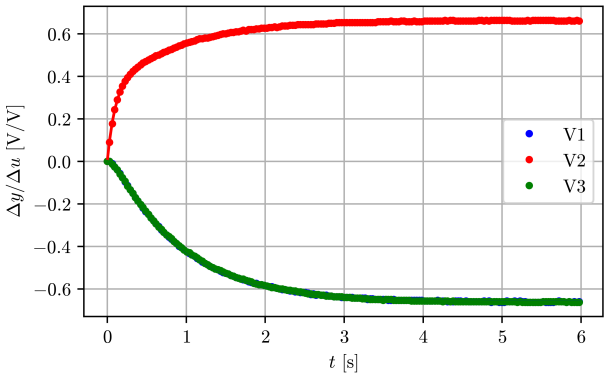

Figure 4: y_1, y_2, y_3 response to $u_2 = 0 \to 3$ V step input

The $\Delta y_1/\Delta u_2$ and $\Delta y_3/\Delta u_2$ pair are essentially the same due to symmetry in the circuit. They could be modeled by a first order system, but there is a slight S-shape in the initial response, so a second order model will be used. This gives,

$$\frac{\Delta y_1(t)}{\Delta u_2(t)} = \frac{\Delta y_3(t)}{\Delta u_2(t)} = K\left[1 + \left(\frac{\tau_1}{\tau_2 - \tau_1}\right)e^{-t/\tau_1} - \left(\frac{\tau_2}{\tau_2 - \tau_1}\right)e^{-t/\tau_2}\right], \tag{9}$$

$$\iff \frac{\Delta Y_1(s)}{\Delta U_2(s)} = \frac{\Delta Y_3(s)}{\Delta U_2(s)} = \frac{K}{(\tau_1 s + 1)(\tau_2 s + 1)}. \tag{10}$$

The $\Delta y_2/\Delta u_2$ pair initially seems like a first order response, but closer inspection shows that the initial slope is too large. The first order model can be multiplied by a lead-lag element to give the faster initial response while having the same settling dynamics. Mathematically,

$$\frac{\Delta y_3(t)}{\Delta u_2(t)} = K\left[1 - \left(\frac{\tau_3 - \tau_1}{\tau_2 - \tau_1}\right)e^{-t/\tau_1} + \left(\frac{\tau_3 - \tau_2}{\tau_2 - \tau_1}\right)e^{-t/\tau_2}\right], \tag{11}$$

$$\iff \frac{\Delta Y_3(s)}{\Delta U_2(s)} = \frac{K\left(\tau_3 s + 1\right)}{\left(\tau_1 s + 1\right)\left(\tau_2 s + 1\right)}. \tag{12}$$

where $0 < \tau_2 < \tau_3 < \tau_1$. The $\tau_2 < \tau_3$ constraint removes the S-shape while the $\tau_3 < \tau_1$ constraint gives the faster initial dynamics. In total, u_2 has a strong interaction with each y_i, but the $u_2 - y_2$ paring has a positive gain and faster dynamics, so it is reasonable to use this pairing.

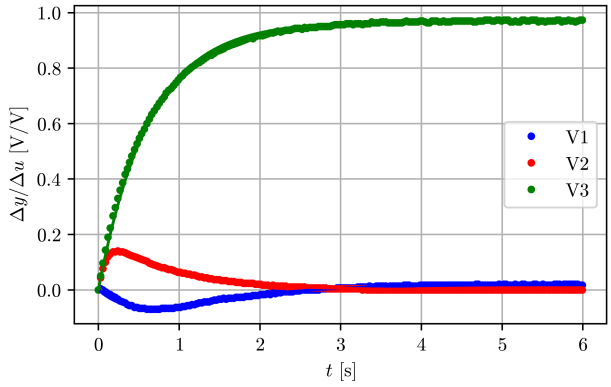

Figure 5: y_1, y_2, y_3 response to $u_3 = 0 \to 3$ V step input

Due to symmetry in the circuit, the u_3 responses are the same as the u_1 responses, just with the roles of y_1 and y_3 swapped. Mathematically, for $u_3 - y_1$,

$$\frac{\Delta y_1(t)}{\Delta u_3(t)} = K\left[1 - \left(\frac{\tau_3 - \tau_1}{\tau_2 - \tau_1}\right)e^{-t/\tau_1} + \left(\frac{\tau_3 - \tau_2}{\tau_2 - \tau_1}\right)e^{-t/\tau_2}\right], \tag{13}$$

$$\iff \frac{\Delta Y_1(s)}{\Delta U_3(s)} = \frac{K\left(\tau_3 s + 1\right)}{\left(\tau_1 s + 1\right)\left(\tau_2 s + 1\right)}. \tag{14}$$

For the $u_3 - y_2$ pairing,

$$\frac{\Delta y_2(t)}{\Delta u_3(t)} = K\left(e^{-t/\tau_1} - e^{-t/\tau_2}\right) \iff \frac{\Delta Y_2(s)}{\Delta U_3(s)} = K\left[\frac{\tau_1 s}{\tau_1 s + 1} - \frac{\tau_2 s}{\tau_2 s + 1}\right]. \tag{15}$$

And for the $u_3 - y_3$ pairing,

$$\frac{\Delta y_3(t)}{\Delta u_3(t)} = K\left(1 - e^{-t/\tau}\right) \iff \frac{\Delta Y_3(s)}{\Delta U_3(s)} = \frac{K}{\tau s + 1}. \tag{16}$$

Model Summary

To summarize, the entire transfer function matrix is given by,

$$\mathbf{G}(s) = \begin{pmatrix} \dfrac{K_{11}}{\tau_{1,11}s+1} & \dfrac{K_{12}\left(\tau_{1,12}+\tau_{2,12}\right)s}{\left(\tau_{1,12}s+1\right)\left(\tau_{2,12}s+1\right)} & \dfrac{K_{13}\left(\tau_{3,13}s+1\right)}{\left(\tau_{1,13}s+1\right)\left(\tau_{2,13}s+1\right)} \\ \dfrac{K_{21}}{\left(\tau_{1,21}s+1\right)\left(\tau_{2,21}s+1\right)} & \dfrac{K_{22}\left(\tau_{3,22}s+1\right)}{\left(\tau_{1,22}s+1\right)\left(\tau_{2,22}s+1\right)} & \dfrac{K_{23}}{\left(\tau_{1,23}s+1\right)\left(\tau_{2,23}s+1\right)} \\ \dfrac{K_{31}\left(\tau_{3,31}s+1\right)}{\left(\tau_{1,31}s+1\right)\left(\tau_{2,31}s+1\right)} & \dfrac{K_{32}\left(\tau_{1,32}+\tau_{2,32}\right)s}{\left(\tau_{1,32}s+1\right)\left(\tau_{2,32}s+1\right)} & \dfrac{K_{33}}{\tau_{1,33}s+1} \end{pmatrix}. \quad (17)$$

The corresponding model parameters are given as follows.

Table 1: Transfer function model parameters

G_{ij}	K [V/V]	τ_1 [s]	τ_2 [s]	τ_3 [s]
G_{11}	0.97	0.63	-	-
G_{12}	-0.66	0.10	0.88	-
G_{13}	0.02	0.72	0.72	-7.2
G_{21}	0.21	0.83	0.10	-
G_{22}	0.66	0.90	0.10	0.5
G_{23}	0.21	0.82	0.10	-
G_{31}	0.02	0.71	0.71	-7.3
G_{32}	-0.66	0.10	0.89	-
G_{33}	0.97	0.64	-	-

PID Control

One of the simplest control strategies is to use PID control on all loops and initially treat them as SISO loops with interactions acting as disturbances. To verify the input-output pairings discussed in the 'System Identification' section, the relative gain array (RGA) method can be used [1]. The RGA, $\mathbf{\Lambda}$, is calculated as,

$$\mathbf{\Lambda} = \mathbf{G}(0) \otimes \left(\mathbf{G}(0)^{-1}\right)^{\mathrm{T}}, \quad (18)$$

where $\otimes$ represents element-wise multiplication, the superscript T denotes the transpose, and $\mathbf{G}(0)$ is the steady state gain matrix. The inputs and outputs are paired by choosing the non-negative elements closest to 1. Performing these calculations gives,

$$\mathbf{\Lambda} = \begin{pmatrix} 1.00 & 0.00 & 0.00 \\ 0.00 & 1.00 & 0.00 \\ 0.00 & 0.00 & 1.00 \end{pmatrix}. \quad (19)$$

The RGA is essentially the identity matrix up to the accuracy of the data. This means that there are almost no steady state interactions when the inputs are paired along the diagonals

(1-1, 2-2, 3-3). This makes physical sense because these components are close to each other and most of the interactions are transient, which the RGA does not account for.

With the pairings identified, the controllers can be tuned by assuming that the relations for SISO loops hold. For simplicity, only PI controllers are used with the controller gain, K_c, and integral time, τ_I, given by the IMC tuning relations [1]. These parameters were then further refined by empirical testing, where the controllers had to be 'detuned' to be less aggressive and more robust against disturbances. The final tuning parameters are given as follows.

Table 2: PID controller parameters

Controller	K_c [-]	τ_I [s]
PI_1	1.46	0.32
PI_2	0.75	0.25
PI_3	1.46	0.32

The controllers are implemented using the discrete PID algorithm [1],

$$\Delta u_k = K_c \left[(e_k - e_{k-1}) + \frac{\Delta t}{\tau_I} e_k \right], \tag{20}$$

where Δt is the sampling time. e_k is the control error, defined as,

$$e_k = y_k^r - y_k, \tag{21}$$

where y_k^r is the reference output value and y_k is the measured output value. To prevent integral windup, if u_k is at the saturation limits, then Δu_k is set to 0 to prevent accumulation of error.

PID Setpoint Tracking

If capacitor voltages are akin to level control, then setpoint tracking is generally a less important objective than disturbance rejection. However, setpoint tracking is still a common benchmark for testing control systems, so it is presented in the following section. To test each y_i, the system was initialized at 0 V, moving to a 3 V setpoint at time 0, and then decreasing to a 2 V setpoint after 2 seconds. The large jump from 0 to 3 V verifies that the main variable gets to the setpoint quickly while suppressing interactions with the other outputs. The 3 to 2 V jump ensures the same, but when the system is moving in the opposite direction. The responses of the output variables are shown in the following figures.

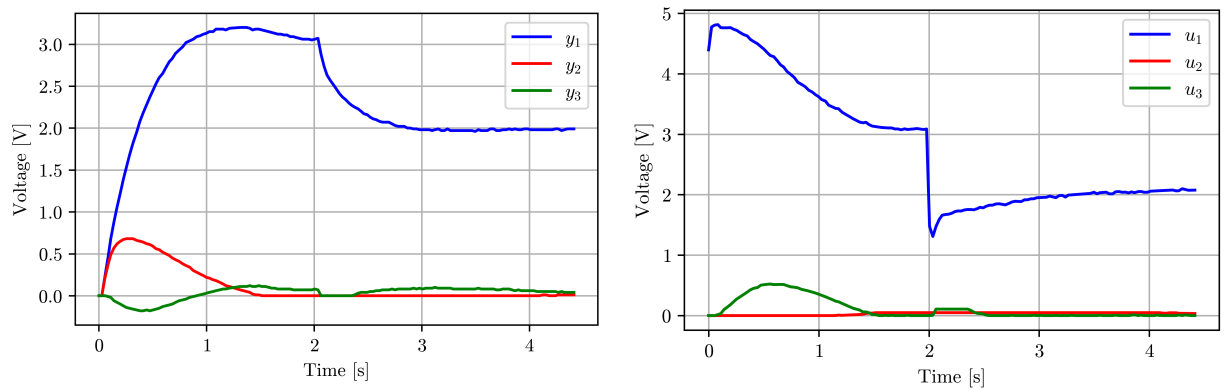

Figure 6: PID controller response to a $y_1 = 0 \rightarrow 3 \rightarrow 2$ V setpoint change; [Left] Control output; [Right] Control input

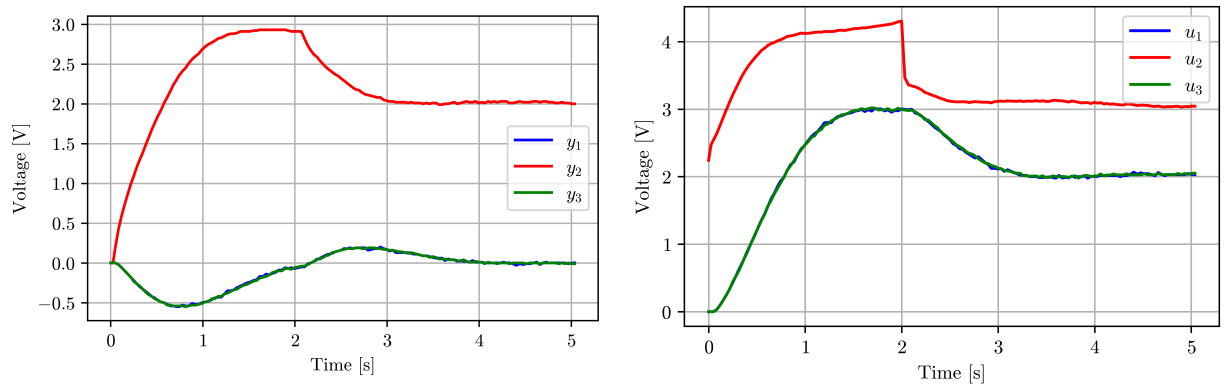

Figure 7: PID controller response to a $y_2 = 0 \rightarrow 3 \rightarrow 2$ V setpoint change; [Left] Control output; [Right] Control input

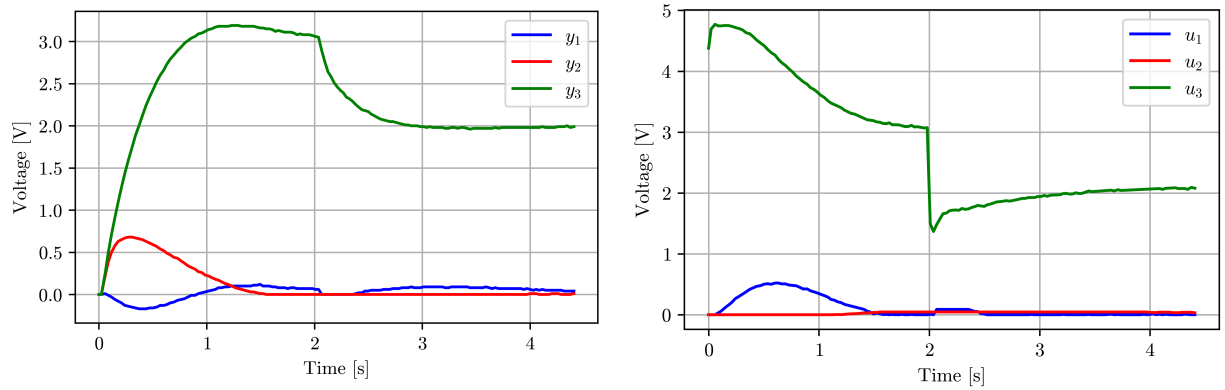

Figure 8: PID controller response to a $y_3 = 0 \rightarrow 3 \rightarrow 2$ V setpoint change; [Left] Control output; [Right] Control input

The y_1 and y_3 setpoint responses are almost the same due to the symmetry of the circuit. For y_1 setpoint change, the rise time is about 0.8 s with a slight overshoot from the 3 V setpoint. The response settles out at about 2 seconds, after which the downwards setpoint change occurs. It takes about 1 second to reach this new setpoint, but there is almost no overshoot. The y_2 voltage shows a large deviation of 0.7 V, but is suppressed quickly and remains at 0 V after 1.5 seconds. Conversely, the y_3 voltage exhibits smaller deviations, but they persist for longer; by the time the second setpoint change occurs, the y_3 voltage has not returned to 0 V. This is because the control input is at its minimum and thus has to rely on the slower natural dynamics of the system to return back to 0 V. The control inputs are also very abrupt, most clearly seen at 2 seconds, where there is an immediate drop in the input voltage to match the setpoint. It will be shown that MPC has smoother inputs, which can be less taxing on the hardware.

The y_2 setpoint tracking is similar to y_1 and y_3, with a quick rise time and no overshoot, reaching the 3 V setpoint in about 2 seconds. The downward step again has a 1 second settling time and no overshoot. The more interesting dynamics are with the y_1 and y_3 interactions, which are large and settle out slowly. When the first setpoint change occurs, it takes the full 2 seconds to recover these variables back to 0 V and when the second setpoint change occurs, it takes another 2 seconds to settle again. This is only slightly faster than the open loop response (~ 2.8 s) which highlights one of the main drawbacks of PID control. For y_1 and y_3 setpoint changes, the interactions are small and can be handled well by PID, but for the y_2 setpoint change, the interactions are too large to easily decouple.

PID Disturbance Rejection

Disturbance rejection is often the more important objective for capacitors and level control, so testing large disturbances is key for robust control design. Disturbances will be implemented as random 'shocks' in the source voltages, specifically, u_3 will have an extra noise term, d, added. The magnitude of these shocks are sampled from a positive Gaussian random variable with a mean of 0 V and standard deviation of 0.3 V. The duration of the shocks are 10 sampling periods (~ 0.3 s). Overall, this gives large and frequent deviations, which serves as a more extreme test for disturbance rejection. The y_3 setpoint will be changed in the same manner as the previous section, going from 0 to 3 V for 2 seconds and 3 to 2 V afterwards.

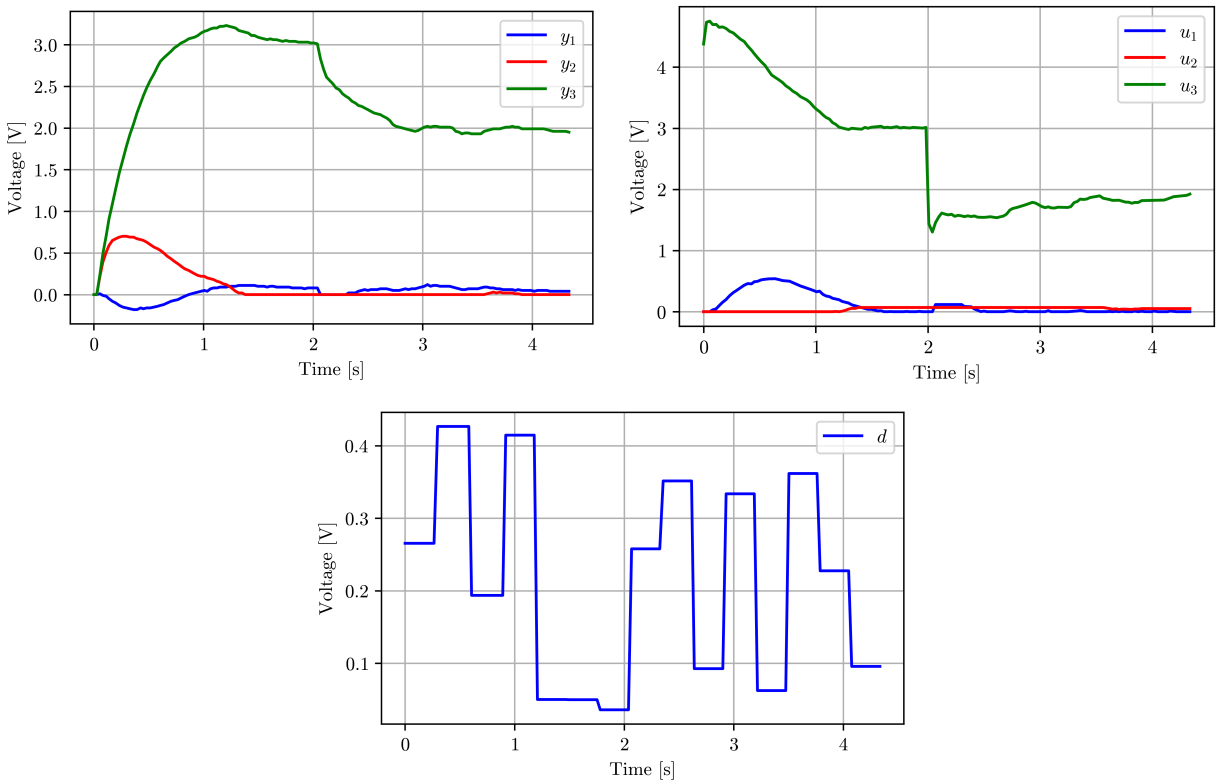

Figure 9: PID controller response to a $y_3 = 0 \rightarrow 3 \rightarrow 2$ V setpoint change; [Left] Control output; [Right] Control input; [Bottom] u_3 disturbance input

The PID controller responds adequately to the random shocks, being able to keep the setpoint of all variables, even with frequent 0.4 V jumps. However, the controllers were tuned to handle these responses because when the initial IMC parameters were used, the controller completely failed and saturated the u_3 input. This was fixed by 'detuning' the loops to be less aggressive with smaller K_c values, making the controller less sensitive. The τ_I values were also decreased to keep a small settling time. This slows the setpoint tracking response, but makes the disturbance rejection more robust by limiting the sensitivity of the controller to large shocks. While the detuned controllers are less sensitive, they still exhibit rigid control inputs, most clearly seen with the setpoint drop at 2 seconds.

Overall, PID control works well for systems with small interactions, but it becomes more difficult to use with large interactions or frequent disturbances. Interactions and disturbances are more sensitive to the tuning parameters used and thus the controllers must be less aggressive to prevent immediate saturation. This limits the response time, but makes the controller more robust and able to handle both setpoint tracking and disturbance rejection.

Model Predictive Control

From the previous section, PID control struggles to handle large interactions and frequent disturbances. Model predictive control aims to limit interactions by predicting the future

response of the system and optimizing control moves to minimize the response error. Disturbances are still not measured since MPC is a feedback control scheme, but they are handled more smoothly since disturbance interactions are directly accounted for. Constraints can also be explicitly enforced in the MPC optimization step, but this was not explored in this report.

The MPC scheme is explained in more detail in the appendix, but the unconstrained control law can be simplified to [1],

$$\Delta \mathbf{U}_k = \underbrace{\left(\mathbf{S}^{\mathrm{T}}\mathbf{Q}\mathbf{S} + \mathbf{R}\right)^{-1}\mathbf{S}^{\mathrm{T}}\mathbf{Q}}_{\mathbf{K}_c}\underbrace{\left(\mathbf{Y}_k^r - \mathbf{Y}_k^{UF} - [\mathbf{Y}_k^m - \mathbf{Y}_k]\right)}_{\mathbf{E}_k}, \tag{22}$$

where $\Delta\mathbf{U}_k$ is the predicted control move vector, $\mathbf{S}$ is the step response matrix, $\mathbf{Q}$ is the output weighting matrix, $\mathbf{R}$ is the input weighting matrix, $\mathbf{Y}_k^r$ is the reference signal vector, $\mathbf{Y}_k^{UF}$ is the unforced response vector, $\mathbf{Y}_k^m$ is the measured output vector, and $\mathbf{Y}_k$ is the predicted output at time k. The $\mathbf{Y}_k^{UF}$ term accounts for past control moves and the $\mathbf{Y}_k^m - \mathbf{Y}_k$ term accounts for model errors and disturbances. In total, without constraints, MPC boils down to simple feedback control,

$$\Delta \mathbf{U}_k = \mathbf{K}_c\mathbf{E}_k, \tag{23}$$

where $\mathbf{K}_c$ is tuned through the model horizon, M, prediction horizon, P, and weighting matrices, $\mathbf{Q}$ and $\mathbf{R}$. The constrained problem has no closed form solution and must be solved through quadratic programming. For simplicity, the optimal solution will be approximated by calculating the unconstrained problem and setting the inputs to the minimum or maximum values if they violate the constraints.

The model and prediction horizons are selected based on the total response time, N, of the open loop system. N is approximated by dividing the response time (~ 3.15 s) by the sampling time, $\Delta t = 0.03$ s, giving $N = 105$. Two cases were tested, the first being an aggressive controller with $M = 0.2N = 21$, $P = 0.3N = 31$, $\mathbf{Q}$ as the identity matrix, and $\mathbf{R}$ as the 0.1 times the identity matrix. The small P makes the predictions occur over a smaller period, forcing a faster response, while the 0.1 multiplier on $\mathbf{R}$ gives minimal suppression on the control moves. The second case was a less aggressive controller with $M = 0.2N = 21$, $P = 0.5N = 52$, $\mathbf{Q}$ as the identity matrix, and $\mathbf{R}$ as the 10 times the identity matrix. The larger prediction horizon allows control moves to occur over a longer period and the 10 weighting on $\mathbf{R}$ suppresses large control moves. These tuning parameters are summarized in the following table. For brevity, only the less aggressive case 2 will be discussed since it gave better results, but the results for case 1 may still be found in the appendix.

Table 3: MPC tuning parameters

Case	N	M	P	$\mathbf{Q}$	$\mathbf{R}$
1	105	21	31	1	0.1
2	105	21	52	1	10

MPC Setpoint Tracking

The setpoint tests are the same as for the PID controllers, the system was initialized at 0 V, moving to a 3 V setpoint at time 0, and then decreasing to a 2 V setpoint after 2 seconds. The large jump from 0 to 3 V verifies that the main variable gets to the setpoint quickly while suppressing interactions with the other outputs. The 3 to 2 V jump ensures the same, but when the system is moving in the opposite direction. The responses of the output variables are shown in the following figures.

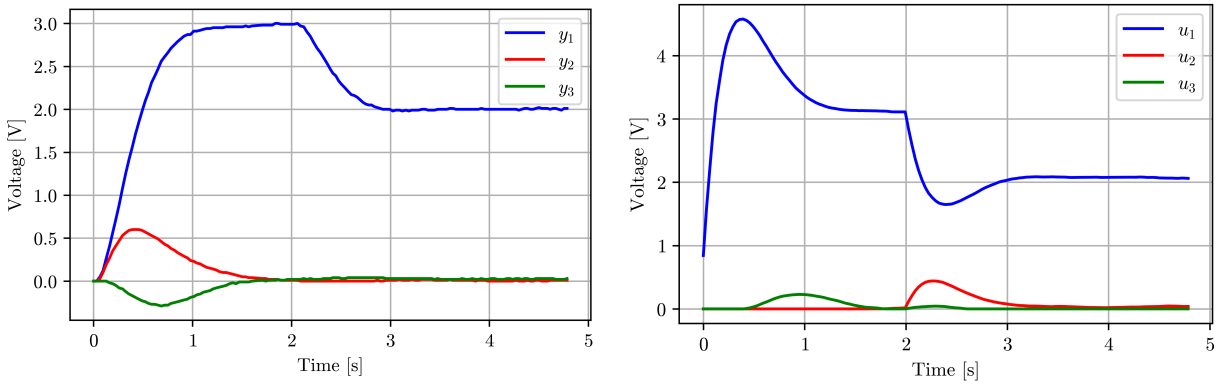

Figure 10: MPC response to a $y_1 = 0 \rightarrow 3 \rightarrow 2$ V setpoint change; Case 2; [Left] Control output; [Right] Control input

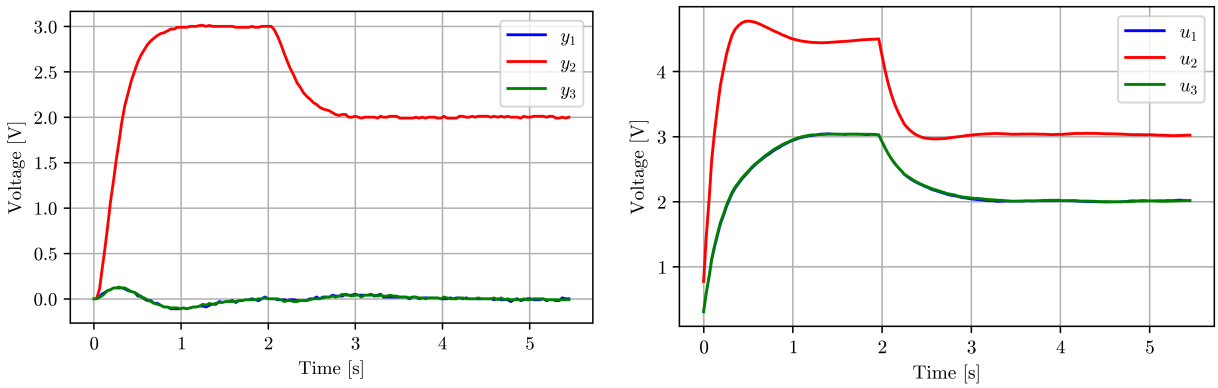

Figure 11: MPC response to a $y_2 = 0 \rightarrow 3 \rightarrow 2$ V setpoint change; Case 2; [Left] Control output; [Right] Control input

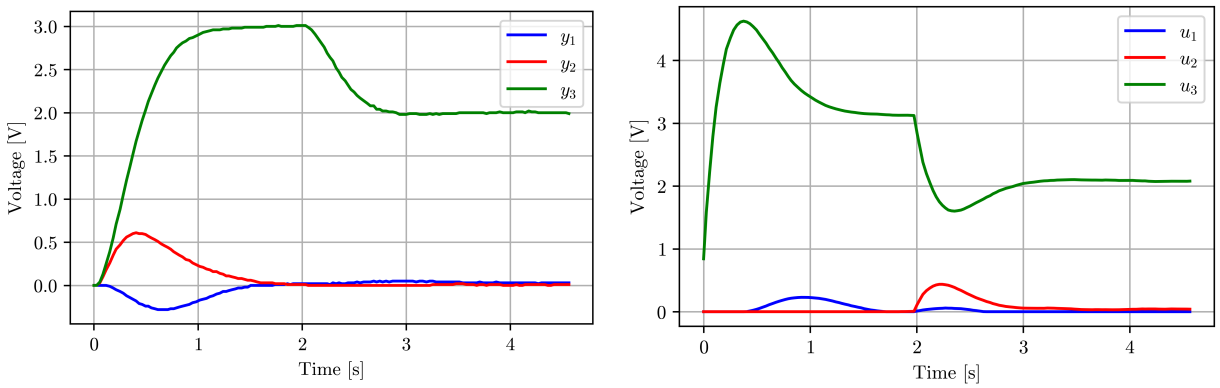

Figure 12: MPC response to a $y_3 = 0 \to 3 \to 2$ V setpoint change; Case 2; [Left] Control output; [Right] Control input

The y_1 and y_3 setpoint responses are again the same due to the symmetry of the circuit. For y_1 setpoint change, the rise time is about 1 s with no overshoot from the 3 V setpoint. The response settles out at about 1 second, after which the downwards setpoint change occurs. It takes about 1 second to reach this new setpoint with no overshoot. The y_2 voltage shows a large deviation of 0.6 V, but is suppressed quickly and remains at 0 V after 1.6 seconds. The y_3 voltage exhibits smaller deviations, about -0.25 V but settles to 0 V after 1.6 seconds. The control inputs are also much smoother than PID control, with no abrupt jumps in the voltage sources.

The y_2 setpoint tracking is similar to y_1 and y_3, with a quick rise time and no overshoot, reaching the 3 V setpoint in about 1 second. The downward step again has a 1 second settling time and no overshoot. The y_1 and y_3 interactions are much smaller than seen with the PID controller. The MPC deviations hardly exceed 0.25 V while PID gave deviations over 0.5 V, however, the length of the deviations are still about 2 seconds. The control inputs are again smoother for MPC compared to the abrupt control inputs by PID.

Overall, MPC performs better than PID since the setpoint is achieved quicker, there is no overshoot, interactions are suppressed quicker, and the control inputs are smoother. Large interactions are handled better by MPC since it explicitly predicts these interactions while PID implicitly treats interactions as disturbances. Despite MPC performing better, the performance gain is only marginally better than PID and the computational costs are significantly higher. Therefore, MPC should only be used on control loops where that slight performance benefit is needed or where interactions are high, otherwise PID is much cheaper and performs almost as well with good tuning.

MPC Disturbance Rejection

Disturbances will be implemented in the same way as the PID tests with random shocks added to the u_3 voltage source. Simultaneously, the y_3 setpoint will be changed, going from 0 to 3 V for 2 seconds and 3 to 2 V afterwards. Shown first is the case 1 (aggressive) controller, which fails after saturating the u_3 input.

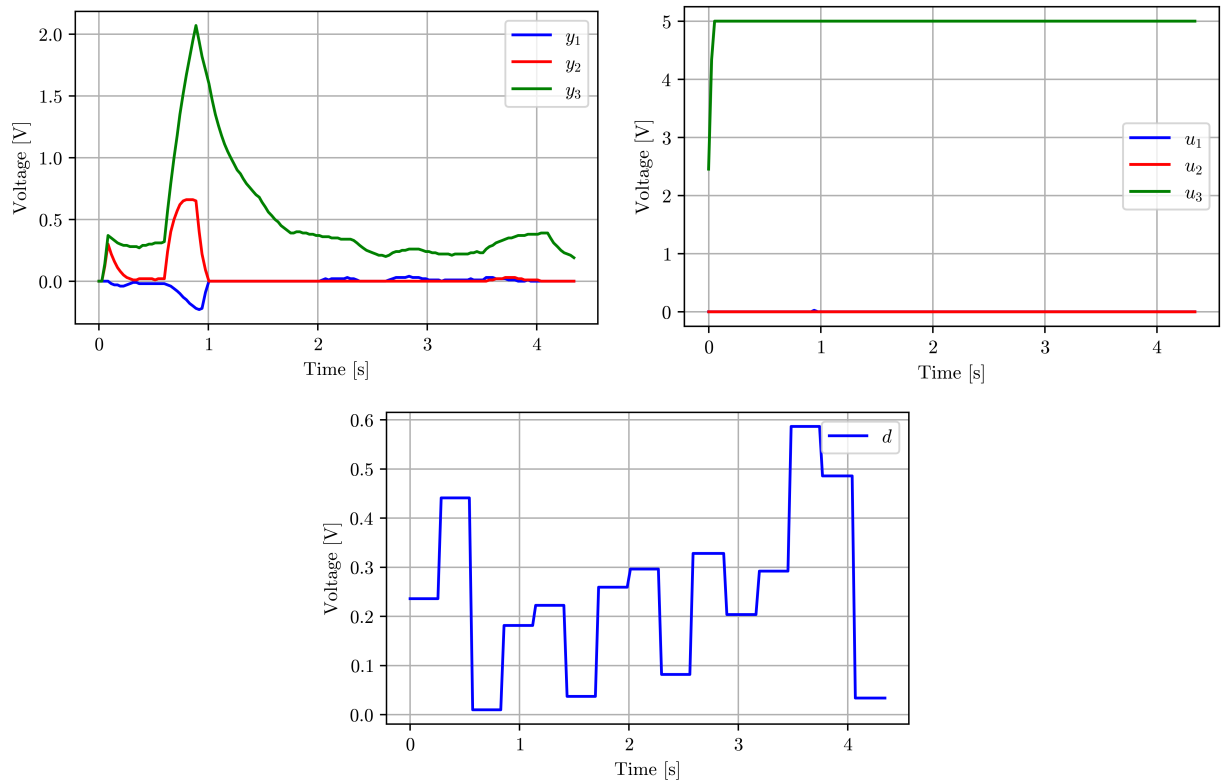

Figure 13: MPC response to a $y_1 = 0 \rightarrow 3 \rightarrow 2$ V setpoint change; Case 1; [Left] Control output; [Right] Control input; [Bottom] u_3 disturbance input

The controller fails mainly due to the small move suppression matrix, $\mathbf{R} = 0.1 \times \mathbf{I}$. This allows the control moves to be much larger than practical and immediately saturates the controller. Additionally, the small $\mathbf{R}$ means that the inverse operation in the gain matrix, $\left(\mathbf{S}^{\mathrm{T}}\mathbf{Q}\mathbf{S} + \mathbf{R}\right)^{-1}\mathbf{S}^{\mathrm{T}}\mathbf{Q}$, can become ill-conditioned. This means that small model errors lead to very different $\mathbf{K}_c$ matrices and predictions become inaccurate. When there is a model mismatch, such as with a disturbance, the ill-conditioned $\mathbf{K}_c$ matrix amplifies these errors and causes the predictions to be completely wrong. This gives the erratic behavior shown in the previous figure. To remedy this, case 2 uses $\mathbf{R} = 10 \times \mathbf{I}$, which limits the size of the control moves and ensures that $\left(\mathbf{S}^{\mathrm{T}}\mathbf{Q}\mathbf{S} + \mathbf{R}\right)^{-1}$ is invertible since $\mathbf{R}$ is full rank and diagonally dominant. This makes the condition number of $\mathbf{K}_c$ smaller and small deviations from the model (disturbances) are not amplified as severely. The results for case 2 are shown in the following figure.

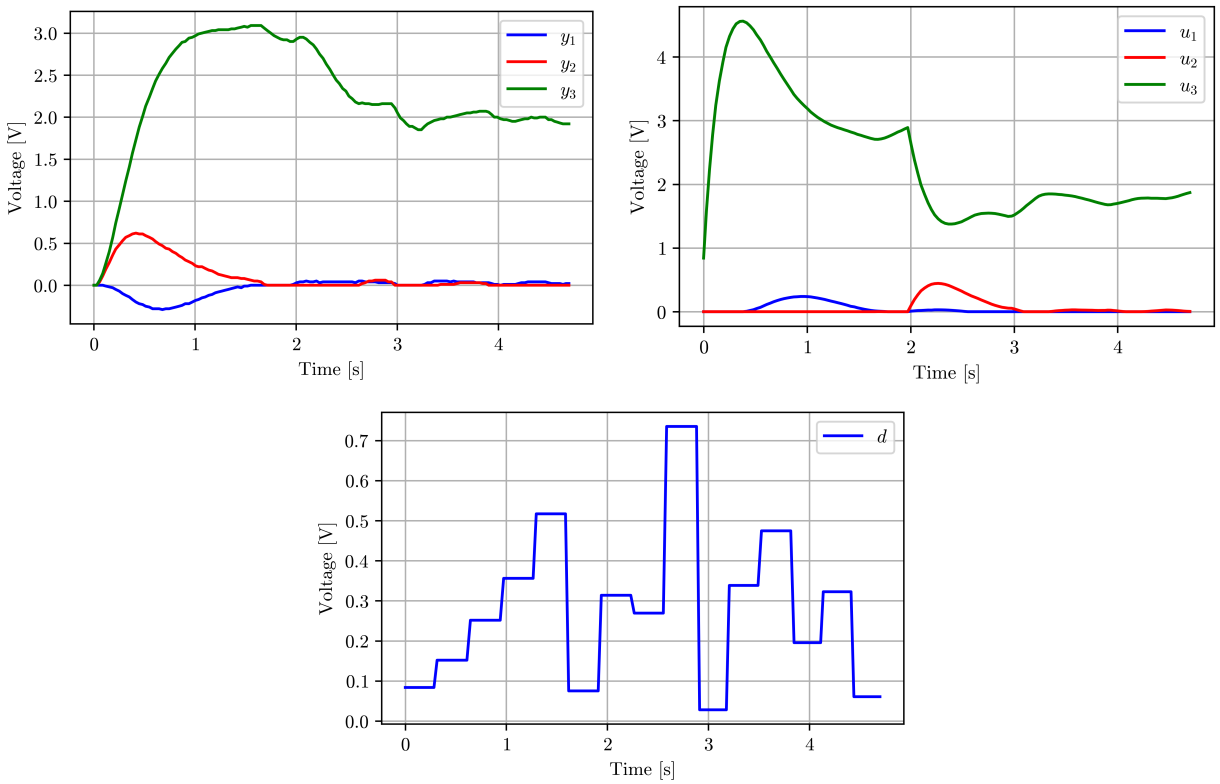

Figure 14: MPC response to a $y_1 = 0 \rightarrow 3 \rightarrow 2$ V setpoint change; Case 2; [Left] Control output; [Right] Control input; [Bottom] u_3 disturbance input

MPC responds well to the random shocks, being able to keep the setpoint of all variables, even with up to 0.7 V jumps. The MPC performance is again only marginally better than PID, giving the similar response times, and slightly less variance in the controlled variables. However, the control inputs of MPC are much smoother than PID, which significantly changes the manipulated variables once each shock hits. MPC only changes the MV's significantly when the setpoints change, seen with the large bumps. The input changes due to the random shocks are barely noticeable for u_1 and u_2, indicating good decoupling.

In total, MPC offers marginal performance gains over PID at much greater computational costs. Generally, the variance of the controlled variables is smaller, allowing the setpoint to be set closer to the operating limits. Additionally, the control inputs are smoother than PID, which is often better for reducing ware on physical equipment. If reducing the variance of the CV's significantly saves costs, then MPC would be an ideal option since it offers better performance. However, if a slightly larger variance is acceptable, then a well-tuned PID loop still offers good performance at a much cheaper cost.

Conclusion

The implementation of a MIMO RC circuit on an Arduino board has been demonstrated. The primary objectives of the report have been achieved as an empirical model of the circuit has

been developed and was used to aid the design of a control system for the capacitor voltages. PID and MPC control algorithms were implemented and compared for both setpoint tracking and large disturbance inputs. Overall, the results indicate that MPC offers small benefits over PID for weakly interacting systems and small disturbances. However, for strongly coupled systems, MPC offers much better performance than PID and it is generally more robust for large disturbance rejection.

Another main advantage of MPC is the explicit handling of constraints, but this was not explored in this report. Therefore, future explorations could include the implementation of a quadratic programming algorithm to perform constrained optimization. Additionally, only a small range of tuning parameters for PID and MPC were tested. Testing a wider range of tuning parameters would be beneficial to understand the sensitivity of the control algorithms to those choices. The developed models and code offer a straightforward path for implementation of these explorations and can even generalize for higher order MIMO systems.

References

[1] Seborg, D. E.; Edgar, T. F.; Mellichamp, D. A.; Doyle, F. J. *Process Dynamics and Control*, 4th ed.; Wiley: Hoboken, NJ, 2017.

[2] Marusak, P. M. Advanced Construction of the Dynamic Matrix in Numerically Efficient Fuzzy MPC Algorithms. *Algorithms* 2021, 14 (1), 25. https://doi.org/10.3390/a14010025.

[3] Jang, L.; Lo, R. Spreadsheet Procedure for Simulating Setpoint Tracking in SISO by Dynamic Matrix Control. *Chemical Engineering Education* 2015, 49.

[4] Cutler, C. R. *Dynamic Matrix Control, an Optimal Multivariable Control Algorithm with Constraints*; University of Houston, 1983.

[5] Lundström, P.; Lee, J. H.; Morari, M.; Skogestad, S. Limitations of Dynamic Matrix Control. *Computers & Chemical Engineering* 1995, 19 (4), 409–421.

Appendix 1 - Physical Arduino Setup

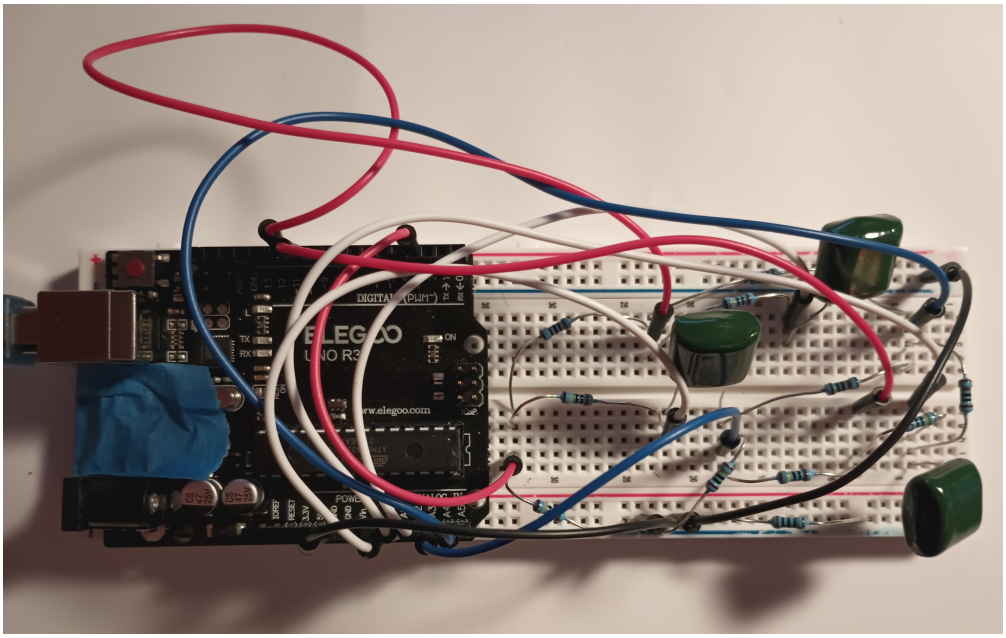

Figure 15: Physical implementation of RC circuit on Arduino board

Appendix 2 - Model Predictive Control Law

Model predictive control requires conversion of the transfer function model to a step response model. This is easily done by inverting the transfer functions to the time domain and discretizing in time. For example, for G_{11},

$$G_{11}(s) = \frac{K}{\tau s + 1} \implies a(t) = \frac{\Delta y_1}{\Delta u_1} = K\left(1 - e^{-t/\tau}\right). \tag{24}$$

For evenly spaced sampling times, Δt, the step response coefficients become,

$$a_i = a(i\Delta t) = K\left(1 - e^{-i\Delta t/\tau}\right). \tag{25}$$

The $u_1 - y_1$ step response matrix, $\mathbf{S}_{11}$, is defined as a $P \times M$ matrix [2,5],

$$\mathbf{S}_{11} = \begin{pmatrix} a_1 & 0 & 0 & \cdots & 0 \\ a_2 & a_1 & 0 & \cdots & 0 \\ a_3 & a_2 & a_1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_P & a_{P-1} & a_{P-2} & \cdots & a_{P-M} \end{pmatrix}. \tag{26}$$

The other $u_j - y_i$ pairings defined similarly. For r inputs and m outputs, the total step response matrix is a $mP \times rM$ block matrix. For this system,

$$\mathbf{S} = \begin{pmatrix} \mathbf{S}_{11} & \mathbf{S}_{12} & \mathbf{S}_{13} \\ \mathbf{S}_{21} & \mathbf{S}_{22} & \mathbf{S}_{23} \\ \mathbf{S}_{31} & \mathbf{S}_{32} & \mathbf{S}_{33} \end{pmatrix}, \tag{27}$$

The prediction vector at time $k+1$, $\mathbf{Y}_{k+1}$, is then defined as [1],

$$\mathbf{Y}_{k+1} = \mathbf{S}\Delta\mathbf{U}_k + \mathbf{Y}_k^{UF} + \mathbf{Y}_k^{bias}, \tag{28}$$

where $\Delta\mathbf{U}_k$ is the vector of predicted control inputs, $\mathbf{Y}_k^{UF}$ is the unforced response, and $\mathbf{Y}_k^{bias}$ is a bias correction term. $\mathbf{Y}_k^{UF}$ accounts for previous control inputs as follows; for a single input-output pair, ij,

$$\mathbf{Y}_{ij,k}^{UF} = \underbrace{\begin{pmatrix} a_{N+1} & a_N & a_{N-1} & \cdots & a_2 \\ a_{N+2} & a_{N-1} & a_{N-2} & \cdots & a_3 \\ a_{N+3} & a_{N-2} & a_{N-3} & \cdots & a_4 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{P+N+1} & a_{P+N} & a_{P+N-1} & \cdots & a_{P+1} \end{pmatrix}}_{\mathbf{S}_{ij}^{UF}} \underbrace{\begin{pmatrix} \Delta u_{j,k-N} \\ \Delta u_{j,k-N+1} \\ \Delta u_{j,k-N+2} \\ \vdots \\ \Delta u_{j,k} \end{pmatrix}}_{\Delta\mathbf{u}_{k,j}}. \tag{29}$$

This is mathematically complex, but it states that $\Delta\mathbf{u}_{k,j}$ is the past N control moves before time k, for input j. $\mathbf{S}_{ij}^{UF}$ propagates the effect of those control moves to time k. The total unforced vector is given by stacking the vectors,

$$\mathbf{Y}_k^{UF} = \begin{pmatrix} \mathbf{S}_{11}^{UF} & \mathbf{S}_{12}^{UF} & \mathbf{S}_{13}^{UF} \\ \mathbf{S}_{21}^{UF} & \mathbf{S}_{22}^{UF} & \mathbf{S}_{23}^{UF} \\ \mathbf{S}_{31}^{UF} & \mathbf{S}_{32}^{UF} & \mathbf{S}_{33}^{UF} \end{pmatrix} \begin{pmatrix} \Delta\mathbf{u}_{k,1} \\ \Delta\mathbf{u}_{k,2} \\ \Delta\mathbf{u}_{k,3} \end{pmatrix} \tag{30}$$

The bias correction, $\mathbf{Y}_k^{bias}$, is defined as the difference between the measured output, $\mathbf{Y}_k^m$ and the predicted output, $\mathbf{Y}_k$,

$$\mathbf{Y}_k^{bias} = \mathbf{Y}_k^m - \mathbf{Y}_k. \tag{31}$$

This term accounts for disturbances and model errors by using actual measured $\mathbf{Y}_k^m$ values. With the predictions for $\mathbf{Y}_{k+1}$, MPC optimizes the control moves to minimize the cost function, J [1,3],

$$J = \mathbf{E}_k^{\mathrm{T}}\mathbf{Q}\mathbf{E}_k + \Delta\mathbf{U}_k^{\mathrm{T}}\mathbf{R}\Delta\mathbf{U}_k, \tag{32}$$

where $\mathbf{E}_k = \mathbf{Y}_k^r - \mathbf{Y}_k$ is the error vector and $\mathbf{Q}$ and $\mathbf{R}$ are weighting matrices. $\mathbf{Q}$ scales the importance of output variables while $\mathbf{R}$ is essentially a Lagrange multiplier which limits the size of control moves. The unconstrained solution is found by setting the gradient of the problem to 0,

$$\frac{\partial J}{\partial \Delta\mathbf{U}_k} = \mathbf{0}, \tag{33}$$

giving the solution,

$$\Delta\mathbf{U}_k = \underbrace{\left(\mathbf{S}^{\mathrm{T}}\mathbf{Q}\mathbf{S} + \mathbf{R}\right)^{-1}\mathbf{S}^{\mathrm{T}}\mathbf{Q}}_{\mathbf{K}_c} \underbrace{\left(\mathbf{Y}_k^r - \mathbf{Y}_k^{UF} - [\mathbf{Y}_k^m - \mathbf{Y}_k]\right)}_{\mathbf{E}_k^{UF}}. \tag{34}$$

Therefore, for unconstrained MPC, the gain matrix, $\mathbf{K}_c$ can be calculated once and only a simple matrix vector multiplication is required in real time. Constraints can significantly

increase the complexity of the problem, but they can be approximated by solving the unconstrained problem first and applying the constraints afterwards. This does not necessarily give an optimal solution, but is sufficient for the simple system in this report.

Implementation of the control algorithm can be seen in the provided Python script. The gain matrix is calculated offline while the optimal control moves are calculated in real time. In summary, the Arduino reads the current capacitor voltages, sends them to the Python script which calculates the control moves, the script sends the inputs back to the Arduino, which then sets the source voltages. This is repeated until the maximum time iterations have been reached.

The block structure of the $\mathbf{K}_c$, $\mathbf{S}^{UF}$, and $\mathbf{S}$ matrices can be visualized in the following figures which plot the magnitude of the matrix elements.

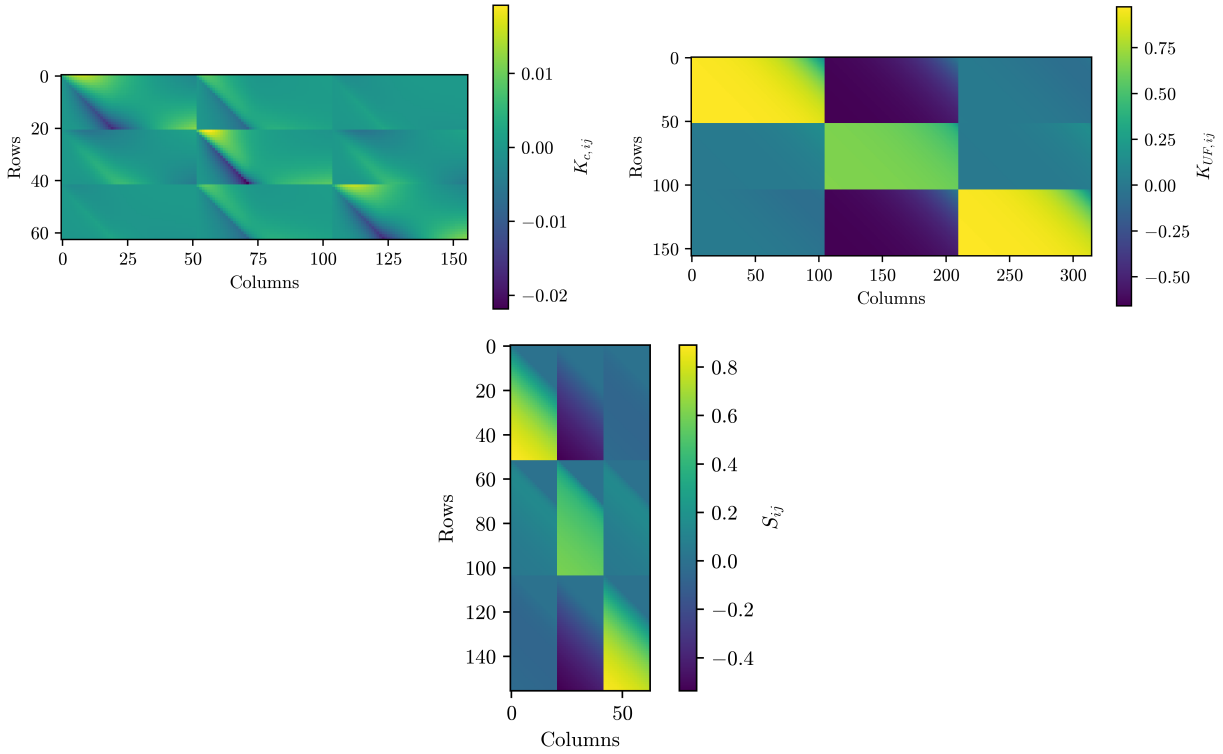

Figure 16: [Left] $\mathbf{K}_c$ block matrix; [Right] $\mathbf{S}^{UF}$ block matrix; [Bottom] $\mathbf{S}$ block matrix

Appendix 3 - Step Response Verification Data

The following figures are verification step responses for the empirical fitting of the transfer function model. The dots denote the measured data while the continuous lines denote the model fit.

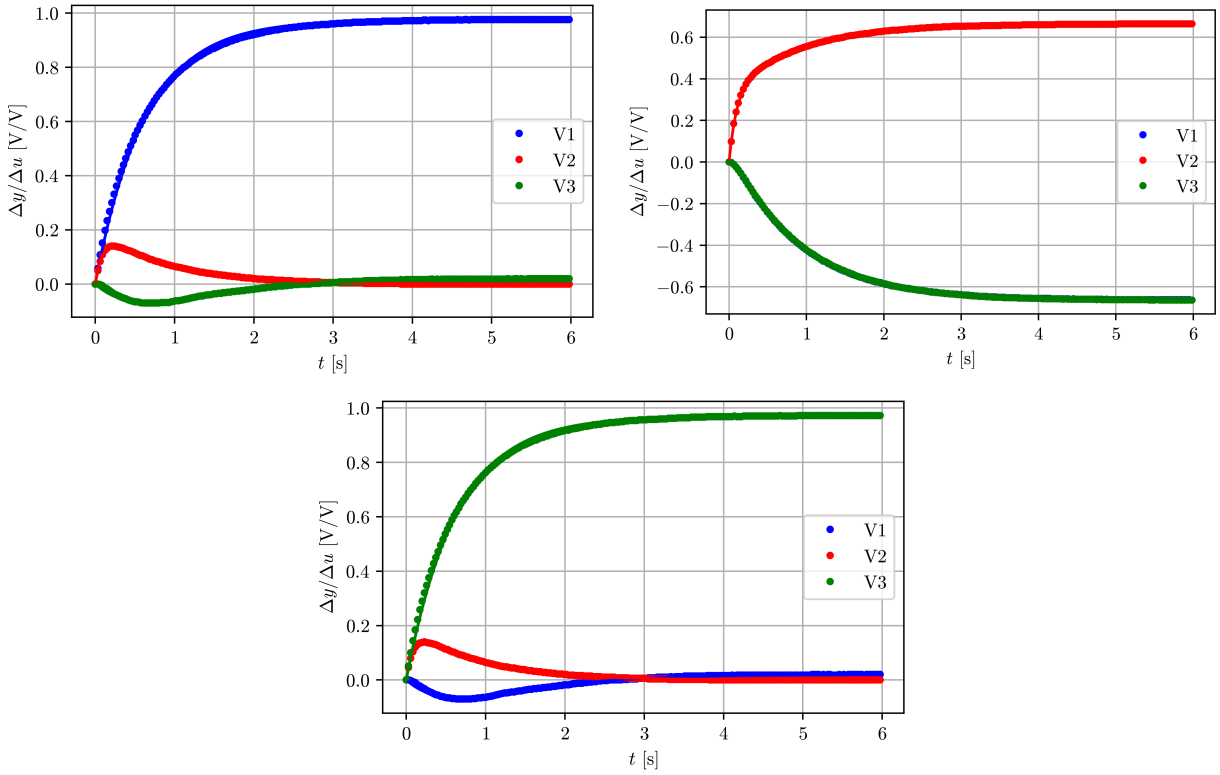

Figure 17: [Left] y_1, y_2, y_3 response to $u_1 = 0 \to 5$ V step input; [Right] y_1, y_2, y_3 response to $u_2 = 0 \to 5$ V step input; [Bottom] y_1, y_2, y_3 response to $u_3 = 0 \to 5$ V step input

Appendix 4 - Extra MPC Data

The following figures are results for the case 1 (aggressive) MPC tuning parameters.

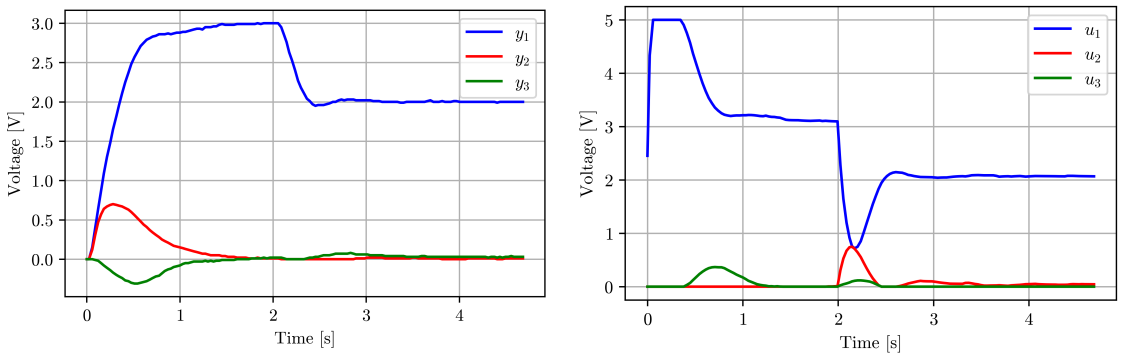

Figure 18: MPC response to a $y_1 = 0 \rightarrow 3 \rightarrow 2$ V setpoint change; Case 1; [Left] Control output; [Right] Control input

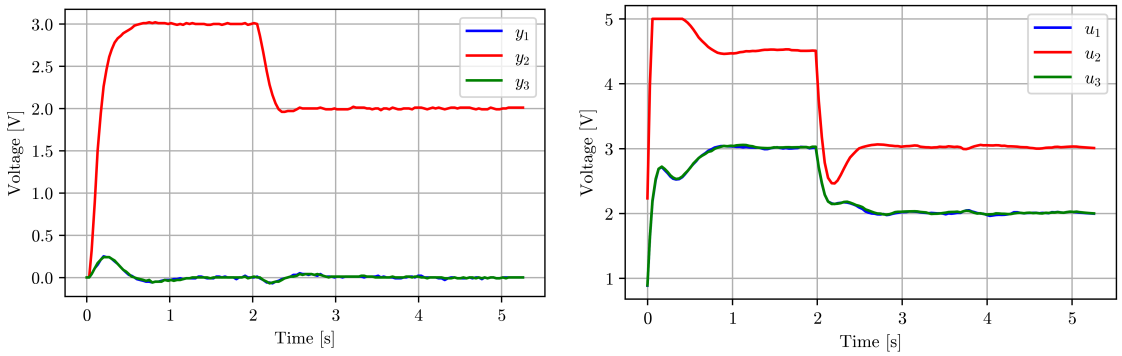

Figure 19: MPC response to a $y_2 = 0 \rightarrow 3 \rightarrow 2$ V setpoint change; Case 1; [Left] Control output; [Right] Control input

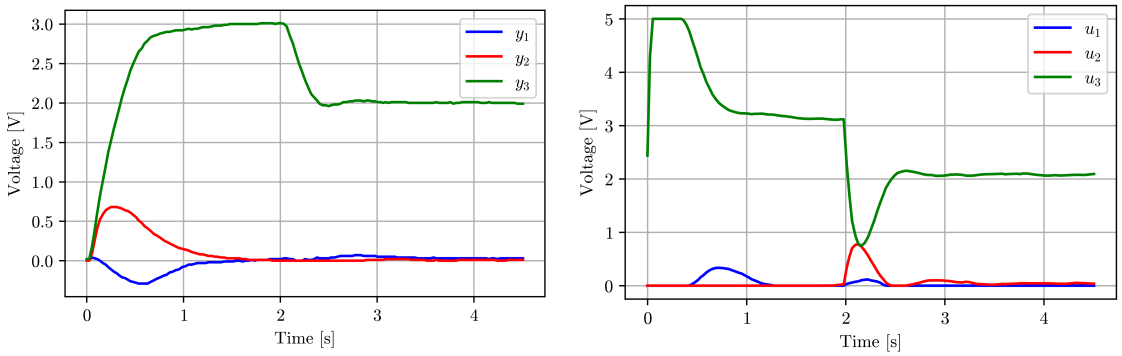

Figure 20: MPC response to a $y_3 = 0 \rightarrow 3 \rightarrow 2$ V setpoint change; Case 1; [Left] Control output; [Right] Control input